T01 Top-Down Movement

CRE132 worksheet

What you'll make

A player who walks in eight directions across a meadow seen from above, like in *Zelda* or *Stardew Valley*. You'll write one short movement script and see how the scene stops the player walking through trees, rocks, fences and the pond.

By the end you'll know how a top-down game differs from a platformer (no gravity, and `y` now means up and down the screen), why diagonal movement needs fixing, and why only the feet bump into things.



Figure 1: The finished meadow: crossing paths, trees, bushes and a rock, with the player on the path.

Before you start

1. Unzip `CRE132_T01_TopDownMovement_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T01_TopDownMovement_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T01_TopDownMovement`.
3. Press **Play**. The player stands on the path playing its idle animation, but the keys do nothing yet. Press Play again to stop.

The scene has a **Grid** with the **Ground**, **Water**, **Fences** and **Props** tilemaps, trees, rocks and bushes, a **CM Follow Camera** that follows the **Player**, and the **Player** itself with C07's **PlayerAnimator**. You'll give the player a way to move.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Switching gravity off

Select the **Player** and look at its **Rigidbody 2D**. It's already set up for a top-down game:

- **Gravity Scale** is **0**. In a platformer, gravity pulls the player down. Seen from above, "down" is just the bottom of the screen, so with gravity on the player would slide off it.
- **Constraints > Freeze Rotation Z** is ticked, so bumping a corner doesn't spin the player round.
- **Interpolate** is set to **Interpolate**, so the movement looks smooth even though physics only updates 50 times a second.

Test it: set **Gravity Scale** to 1, press Play and watch the player fall down the screen. Set it back to **0**.

2. The TopDownMovement script

Make a folder `Assets/_game/Scripts/T01_TopDownMovement` and create a script in it called `TopDownMovement`:

```
// Moves a top-down character in eight directions with the Input System.
// The Rigidbody2D has Gravity Scale 0, so nothing pulls it down the screen.
using UnityEngine;
using UnityEngine.InputSystem;

public class TopDownMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 4f;

    // Keeps diagonal movement the same speed as straight movement. The Input System's
    // WASD composite already does this (its Mode is Digital Normalized); to see the
    // problem, set that Mode to Digital in the Input Actions asset and untick this.
    [SerializeField] private bool normaliseDiagonals = true;

    [SerializeField] private InputActionReference moveAction;

    private Rigidbody2D body;
    private Vector2 moveInput;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void OnEnable()
    {
        moveAction.action.Enable();
    }

    private void OnDisable()
    {
        moveAction.action.Disable();
        // Stop when switched off (for example while attacking), or the body would keep
        // sliding.
        if (body != null)
        {
            body.linearVelocity = Vector2.zero;
```

```
        }
    }

    private void Update()
    {
        moveInput = moveAction.action.ReadValue<Vector2>();

        // Up + Right on a keyboard gives (1, 1), which is 1.41 long, so diagonals would
        // be faster. Normalising makes it 1 long. Only longer inputs are shortened, so a
        // gamepad stick pushed part of the way still moves slowly.
        if (normaliseDiagonals && moveInput.magnitude > 1f)
        {
            moveInput = moveInput.normalized;
        }
    }

    private void FixedUpdate()
    {
        body.linearVelocity = moveInput * moveSpeed;
    }
}
```

What's going on:

- **The Move action gives a `Vector2`:** `x` is left and right, `y` is up and down. A platformer only used `x`; here we use both, so one line, `moveInput * moveSpeed`, moves in any direction.
- **Input is read in `Update`, physics is done in `FixedUpdate`.** `Update` runs every frame, so no key press is missed. `FixedUpdate` runs in step with the physics, which is where a Rigidbody's velocity should be set.
- **`OnDisable` stops the body.** Later samples switch this script off while attacking; without this line the player would keep sliding.

Add it: select the **Player** and click **Add Component > Top Down Movement**. Leave **Move Speed** at **4** and **Normalise Diagonals** ticked. In the **Move Action** slot, click the small circle and pick **Player/Move** (from `CRE132Controls`). Move is bound to WASD, the arrow keys, the gamepad's left stick and the D-pad.

Test it: press Play and walk around with WASD or the arrows. The camera follows you.

3. Why diagonals need fixing

Press **W** and **D** together and the Move action could give (1, 1). That arrow is about 1.41 long, not 1, so moving diagonally would be 41% faster than moving straight. `moveInput.normalized` keeps the direction but makes the length 1.

The Input System already fixes this for the keyboard, so to see the problem you have to switch that off too:

1. Double-click `CRE132Controls` (in `Assets/_game/Input`). In the **Player** map, select **Move**, then the **WASD** composite under it. Change **Mode** from **Digital Normalized** to **Digital**.
2. Select the **Player** and untick **Normalise Diagonals**.
3. Press Play, walk diagonally and look at the **Rigidbody 2D**'s **Info > Linear Velocity** in the Inspector. Straight it's 4; diagonally both numbers are 4, which is faster.

Put both back afterwards: **Mode** to **Digital Normalized** (and save the asset), and **Normalise Diagonals** ticked.

- **Only inputs longer than 1 are shortened.** A gamepad stick pushed halfway gives a short arrow, and the player walks slowly, which is what you want.

4. Bumping into things with your feet

Select the **Player** and look at its **Box Collider 2D**: it's small (**Size** 0.6 by 0.35) and sits at the feet (**Offset Y** −1.0375). Now select a tree: its collider is a small box round the trunk.

- **Only the feet collide.** In a top-down game the bottom of a sprite is where it touches the ground. With a collider at the feet, the player can walk up behind a tree's leaves but stops at the trunk, which looks like depth.
- **The pond and fences are solid too.** The **Water** tilemap has a **Tilemap Collider 2D** with **Composite Operation** set to **Merge**, plus a **Composite Collider 2D** that joins all the water tiles into one smooth shape. The **Fences** tilemap has its own **Tilemap Collider 2D**.

Test it: walk into a trunk, a rock, a fence and the pond. Then walk up behind a tree from below: you'll be drawn on top of its leaves, which looks wrong. T03 fixes that.

Check your work

- ☐ The player walks in eight directions with WASD, the arrows, a gamepad stick or the D-pad, and the camera follows.
- ☐ Diagonal movement is the same speed as straight movement.
- ☐ The player stops at tree trunks, rocks, fences and the pond, but can walk over the top part of a tree.
- ☐ Releasing the keys stops the player straight away.
- ☐ Compare with `CRE132_T01_TopDownMovement_Finished` if something behaves differently.

Key points

- **Top-down is a side view with gravity switched off**, and `y` used for up and down the screen.
- **Normalise diagonal input**, or make sure the input already is, so diagonals aren't faster.
- **Collide with the feet, not the whole sprite.** That's how top-down games fake depth.

If something goes wrong

- **NullReferenceException in TopDownMovement:** the **Move Action** slot is empty.
- **The player falls down the screen:** the Rigidbody 2D's **Gravity Scale** isn't 0.
- **The player spins when it hits a corner:** tick **Freeze Rotation Z** under **Constraints**.
- **The player walks through a tree:** check the tree still has its **Box Collider 2D**, and that it isn't ticked as **Is Trigger**.
- **Diagonals are faster:** you left the WASD composite on **Digital** and unticked **Normalise Diagonals**. Put one of them back.

Try it

1. Add a run button: an `InputActionReference` for C12's **Run** action (Left Shift, or the gamepad's West button) that doubles the speed while held.
2. Make the player slower on grass than on dirt. (Hint: `Tilemap.GetTile` tells you which tile is under the player's feet.)
3. If you have a gamepad, push the stick gently: the player creeps. Why doesn't normalising stop that?